CLB Group

CLB HOLDINGS BERHAD

No 38, Jalan Riang 21, Kawasan Perindustrian Taman Gembira,
81200 Johor Bahru, Johor, Malaysia.
Tel: +607-338 3636
Email: sales@clbgroups.com

Requirement Analysis

Ref. No: PRA/01/260219/001

Project	: JJ Wine	Date	: 19th Feb 2026
Person-in Charge (PIC)	: Khairul Anuar	Department	: Mecha Dept

1 Introduction

1.1 Project Overview

The RFID SDK project aims to develop a **standardized, reader - agnostic RFID Software Development Kit (SDK)** with a PC - based application for monitoring and controlling RFID readers. The SDK serves as a middleware between RFID hardware (e.g., UF3 - S reader) and client systems, providing reliable, event - driven access to RFID tag data, reader status, and diagnostics.

This project replaces previous (voided SDK) Python - based and introduces a modern **NodeJS + React + Electron** stack, providing:

- Cross platform support (windows/Linux)
- Event - driven, asynchronous API for external integration
- Operator - friendly GUI for diagnostics, monitoring, and configuration.

1.2 Purpose

This document provides a comprehensive specification of the **functional** and **non – functional requirements** for the proprietary RFID SDK with its associated Graphical User Interface (GUI). This specification serves as the technical baseline for the development, testing, and stakeholder acceptance.

1.3 Project Objectives

The primary objective is to engineer an RFID SDK with a GUI that supports:

1. To **develop** a **reader – agnostics, Typescript – based** RFID SDK supporting TCP/IP, RS232, and MQTT communication with the reader.
2. To provide a **PC application** (Electron + React) bundling SDK and GUI

2 System Scope & Boundaries

2.1 Product Scope

The system is comprised of two distinct yet integrated modules.

- **RFID SDK (Runtime):** A Node.js/Typescript library responsible for the reader management, transport layer (e.g., TCP/IP, MQTT, RS232), protocol abstraction, event dispatching, etc. for the RFID system
- **Graphical User Interface (GUI):** A cross – platform Electron/React application providing visualization, session management, data format, and export.

2.2 Functional Boundaries

2.2.1 In – Scope

- **Hardware Abstraction:** Pluggable adapter system for multi – vendor support.
- **Connectivity:** Management of TCP/IP, RS – 232, and MQTT socket lifecycles and reconnections.
- **Data Transformation:** Conversion of raw telemetry into JSON and plain text and displayable in the GUI. (Hex, Plain Text, JSON)
- **Security:** Secure IPC bridging between the GUI and SDK layer.
- **Database:** Direct database management with short – term retention period (max 30 days)

2.2.2 Out – of – Scope

- **Firmware:** Modification of reader – side embedded software.
- **Business Logic:** High – level workflows like inventory reconciliation or ERP syncing.

2.2.3 SDK Component

Component	Specification	Description
SDK Core	Node.JS + Typescript	Implement all RFID communication logic, driver abstraction, and event system. Handles TCP/IP, RS – 232, and MQTT communication
Reader Drivers	Typescripts Interface + UF3 – S Driver implementation	Abstract layer allowing for multiple reader types. UF3 – S driver is implemented; new drivers can be added using same interface.
Data Models	Typescript classes	RFIDTags, ErrorModel. Ensures typed data structures for SDK events

No 38, Jalan Riang 21, Kawasan Perindustrian Taman Gembira,
81200 Johor Bahru, Johor, Malaysia.
Tel: +607-338 3636
Email: sales@clbgroups.com

Event System	NodeJS EventEmitter	Provides events: onTagRead, onError, onConnected, onDisconnected, onReaderStatusChange. Support throttling and buffering.
Public API	SDK Methods	Connect(), disconnect(), startReading(), stopReading(), on(event). Promise – based API for consistency
PC App Wrapper	Electron	Combines Node.js SDK + React GUI into a single cross – platform desktop application (.exe)
Communication	TCP/IP, RS – 232, MQTT	Core SDK communication channels to RFID readers. Abstracted to support multiple protocols.
Build Tools	Electron builder, typescript compiler	Bundling, transpilling, and packaging for PC deployment

3 Stakeholder Requirements Overview

Stakeholder	System Usage	Role	Requirement Summary	Priority (H/M/L)
End-User / Client	Real – time monitoring	User	View tag data, export results, system logs, data retention, and cumulative count	H
	SDK library	User	Client develop their own GUI from the functional SDK	H
IT/Technical Team	SDK + GUI	Develop / support	Stable SDK API, extensible architecture, multi – reader support	H
Management/Project Owner	Business analytics	Business Decision Maker	Accurate reporting, export capability.	H

4 Functional Requirements

Requirements that define what the system or solution must perform

Requirement	Description	Criteria
Connection	System **shall** establish connection with the hardware	Successful connection, $t < 5$ secs.
	The system shall allow start/stop scanning function	User – controlled scanning
Data Processing	System shall receive and process tag data in real – time	Tag displayed, $t < 200$ msecs of receptions
Tag Event	The system shall emit a structured tag event	Events include EPC, RSSI, Timestamp.
Real – time Deduplication	The system compute total tag count per session	Accurate cumulative counter (EPC scanned = 5, total tag count = 5)
	The system compute unique EPC count per session	EPC uniqueness is validated
Output Data	System shall support payload in various formats (e.g., JSON, Hex, Plain Text)	User – selectable output format.
Exporting	System shall support CSV/Excel data export	Export file generated successfully
	System Log/system message is able to be exported.	Logs message is exported in .txt format.
Data Retention	System store tag data for up to 30 days.	Data retention configurable for export (1, 7, 30 days)

5 Non-Functional Requirements

Requirements that define system quality, behavior, and constraints.

Requirement	Description	Criteria/Remarks
Performance	High – speed data throughput.	Support ≥ 100 tag reads per seconds and UI remains responsive < 16 ms frame time
Maintainability	Modular architecture	SDK separated from GUI repository.
Security	Secure IPC & Sandboxing	GUI shall communicate with SDK via IPC without modifying SDK logic. Electron context isolation enabled to ensure no direct SDK exposure.
	Secret Management	No hardcoded credentials; or use environment variables or encrypted local storage

Reliability	Error handling & Data integrity	No system crash or malformed/invalid payload displayed
Usability	User interface with localization	English UI with clear tag display and intuitive connection status indicator (connected/disconnected)

6 Workflow Requirements

6.1 Workflow

The workflow will automate RFID data collection and visualization. Refer *Appendix A*

1. RFID reader transmits raw tag data to the RFID SDK
2. The SDK processes and emits structured events
3. The GUI receives data via IPC and displays real – time results
4. The system stores tag data in a local database
5. Users can export data directly to CSV.

7 Data Requirements

7.1 Data Entity: rfid_events

Represent a single RFID tag read event emitted by the SDK

Entity Name	Description
rfid_events	Represent one tag detection event from the RFID reader

7.2 Field Definition Table

Field Name	Data Type	Typescript Type	Default	Validation/Rules
id	INTEGER	Number	AUTOINCREMENT	• Primary Key • Auto – incremented • Must be unique
epc	TEXT	String	-	• Typically hex string • Non – empty string • Case-sensitive depending on usage
reader_id	TEXT	String	-	• Non – empty string • Should match registered reader ID
antenna	INTEGER	Number	-	• Positive integer • Within reader antenna range

Field Name	Data Type	Typescript Type	Default	Validation/Rules
rssi	REAL	Number	-	• Numeric value • Typically negative (-20 to -90 dBm)
read_at	DATETIME	String (ISO)	CURRENT_TIMESTAMP	• Auto – generated • Stored in UTC • Format: YYYY-MM-DD HH:MM:SS

8 System Constraints and Assumptions

The successful of the project depends of several keys assumptions and is bounded by specific technical and operational constraints.

8.1 Assumption

The successful implementation of the RFID SDK project is based on the following key assumptions:

1. **Reader Protocol Availability:**
   The communication protocol specification of the RFID reader (e.g., UF3 – S protocol) is fully documented and accessible to the developer.
2. **Protocol Stability:**
   The reader communication protocol will remain stable during the development lifecycle and will not undergo breaking changes.
3. **Hardware Availability:**
   RFID reader hardware devices are available for development, integration, and validation throughout the project timeline.
4. **Development Resources:**
   All necessary source code, development tools, SDK documentation, and technical references are available to support SDK development.
5. **Operating Environment:**
   The SDK will operate within a supported desktop environment (Windows/macOS) using Electron and Node.js runtime.
6. **Single Reader Active per Session:**
   At initial release, the system assumes one active reader connection per session (multi – reader support may be future enhancement)
7. **Stable Network/Serial Environment:**
   Serial ports/ TCP/IP networks, or MQTT brokers used for communication are assumed to be stable and properly configured.

8. **Database Storage Capacity:**
   The local database will have sufficient storage capacity for at least 30 days of tag event retention period.
9. **User Technical Competency:**
   User operating the application have basic technical knowledge to configure connection parameters (COM port, IP address, MQTT topic, etc.).

8.2 Constrains

The RFID SDK is constrained by the following technical and operational limitations:

1. **Resource Constraints**
   - Limited internal manpower for testing and quality assurance. Testing and validation will be done by the developer.
   - Limited access to multiple RFID reader brands during development. The development only uses UF3 – S 8 – channel reader by SEUIC.
2. **Timeline Constraints**
   - Fixed project timeline: **9th Feb 2026 – 9th Apr 2026**
   - All core features (connect, scan, store, export, etc.) must be completed within this duration. The deliverables will be:
     - SDK + GUI
     - Standalone SDK
3. **Technology Constraints**

   The project must adhere to predefined technology selections:
   - **Programming Language:** Typescript (Node.js environment)
   - **Application Framework:** Electron (Main + Renderer architecture)
   - **User Interface Framework:** React.js
   - **Database:** SQLite (local file – based database)
   - **Communication Protocol (Initial):** UF3 – S Reader protocol
   - **Transport Layer:** Serial (RS – 232), TCP/IP, MQTT
   - **Export Format:** Logs export (.txt), data export (.csv)

4. Architectural Constraints

- The SDK must be **reader – agnostics**, meaning:
  - Reader protocol implementation must be abstracted
  - New reader protocols (e.g., Zebra, Impinj) must be pluggable without core redesign.
- The system must follow:
  - Event – driven architecture (EventEmitter – based)
  - IPC bridge for communication between Electron main and renderer
  - Local – only storage (no cloud backend for this SDK)

5. Performance Constraints

- The system must handle:
  - High – frequency (burst) and long duration tag reads
  - Real – time UI updates.
  - Deduplication and aggregation within session memory.
- Database writes must not block real – time scanning

These assumptions and constrains collectively define the boundaries, resources, and conditions under which the project must be operate, ensuring the objective remain achievable within available capabilities.

9 Key Performance Indicator

For each functional and non – functional requirement, they must fulfill the following KPIs.

- **SDK Functional KPIs**

KPI	Target	Descriptions
Reader connection success rate	≥ 99%	Successful connection/total attempts
Protocol support coverage	100%	TCP/IP, RS – 232, and MQTT fully supported

- **Performance KPIs**

KPI	Target	Descriptions
Tag read throughput	≥ 100 tags/sec	Tag process per minute
Event latency	≤ 50 msecs	Time from tag read → event emission
Start – up time	≤ 5 secs	App launch → ready state
Raw data display latency	≤ 10 secs	Reader scan tag → GUI display

- **Stability & Reliability KPIs**

KPI	Target	Descriptions
Uptime during test	≥ 24 hours	Continuous operation without crash
Auto – reconnect success rate	≥ 98 %	Successful reconnect after disconnect
Crash rate	0 %	Crasher per 3 hours runtime

- **Data Accuracy KPIs**

KPI	Target	Descriptions
Malformed/invalid packet handling	100%	No SDK crash on bad packets

- **PC Application (GUI) KPIs**

KPI	Target	Descriptions
GUI responsiveness	≤ 200 ms	UI response to user action
Real - time display accuracy	100%	GUI reflects SDK state correctly
GUI crash rate	0 %	GUI crashes during operation
Installer success rate	≥ 99%	Successful installation attempts
Cross – platform compatibility	100%	Window & Linux supported
Live Tag data stream	-	Able to display real – time tag data

10 Verification and Validation

To ensure all functional and non-functional requirements meet the specific criteria, the following V&V activities will be carried out:

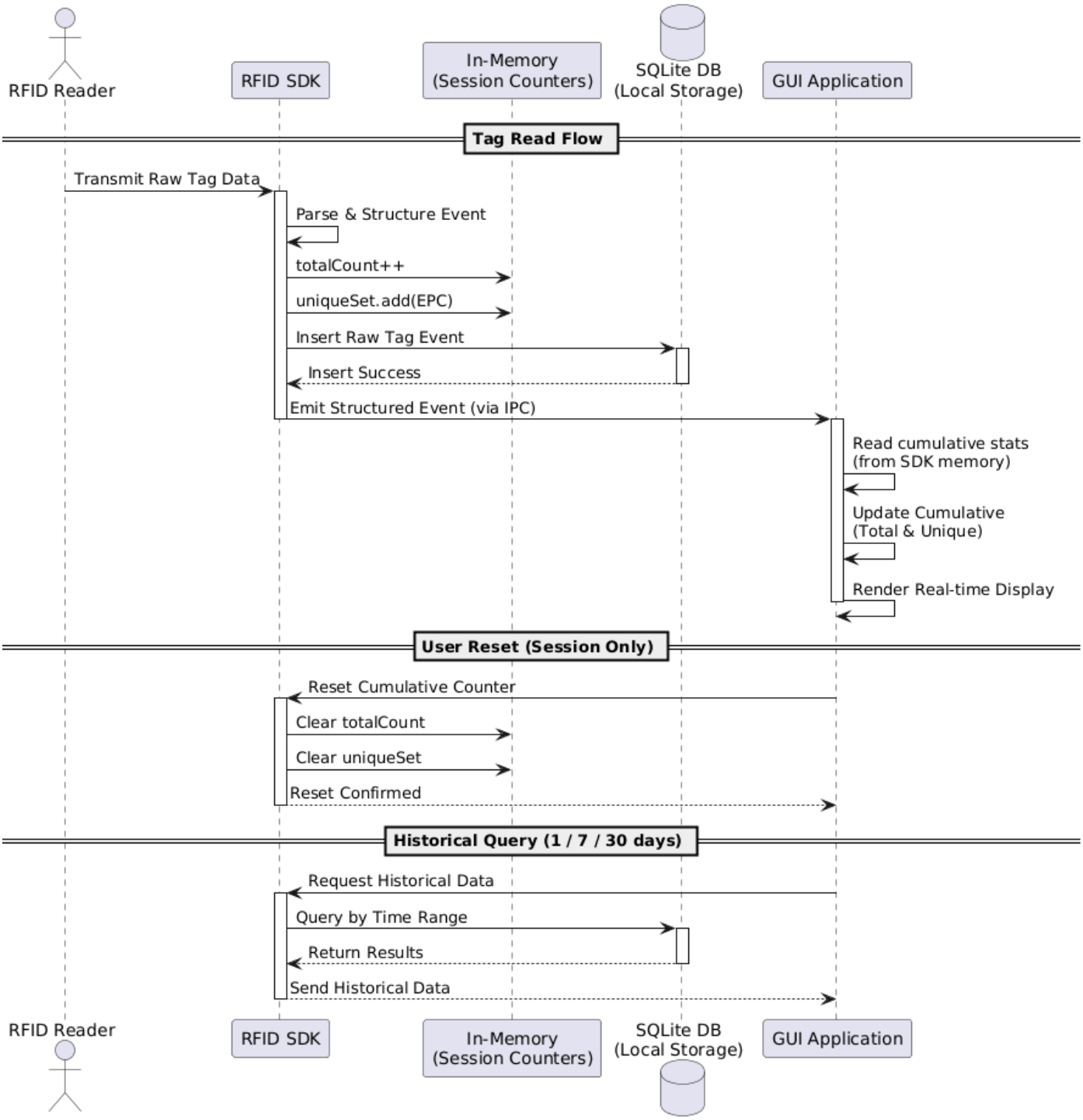
Item	Method	Responsible
SDK API	Unit Testing	Developer
GUI Display	Integration Testing	Developer
Performance	Load testing	Developer
Tag Processing	Functional Testing	QA/Internal Tester

11 Deliverables of Requirement Analysis

These are the expected deliverables for the project along with the timeline.

No.	Deliverable	Dateline	Description
1.	RFID SDK Library	24th Mar 2026	Standalone Node.js SDK
2.	Electron GUI Application	27th Mar 2026	Monitoring & Export tools
3.	Final Deployment Package	9th Apr 2026	Production – ready installer

The undersigned approve the initiation of this project as defined in this document.

Project Manager	**Developer/IT Lead**	**Management**
Name: Khairul Anuar	Name: Tyson	Name Jin Ng
Date: 19th Feb 2026	Date: 23rd Feb 2026	Date 25th Feb 2026

12 Appendix A: RFID Data Collection and Visualization Flow